Chuyên đề: Kỹ thuật Hai Con Trỏ (Two Pointers) & Cửa Sổ Trượt (Sliding Window)

1. Lý thuyết tổng quan

Kỹ thuật hai con trỏ (Two Pointers) là một phương pháp tối ưu hóa cực kỳ mạnh mẽ trong Lập trình thi đấu, thường được sử dụng để giải quyết các bài toán trên mảng (array) hoặc chuỗi (string). Bằng cách duy trì và di chuyển hai biến chỉ mục (gọi là "con trỏ") dọc theo cấu trúc dữ liệu dựa trên một điều kiện nhất định, kỹ thuật này giúp giảm độ phức tạp thời gian từ $O(N^2)$ (hoặc $O(N^3)$) xuống còn $O(N)$ hoặc $O(N \log N)$.

Bản chất của kỹ thuật này dựa trên **tính đơn điệu (monotonicity)** của bài toán. Thay vì xét lại toàn bộ không gian trạng thái, ta tận dụng thông tin từ trạng thái trước đó để cập nhật trạng thái hiện tại.

Về cơ bản, kỹ thuật này được chia thành hai nhóm chính:

1.1. Hai con trỏ ngược chiều

Thường được áp dụng trên các mảng **đã được sắp xếp**. Hai con trỏ được khởi tạo ở hai đầu của mảng ($L = 0$ và $R = N - 1$). Dựa vào tổng, hiệu hoặc một tính chất nào đó của phần tử tại L và R, ta sẽ quyết định tăng L hay giảm R để tiến dần vào giữa.

Đặc điểm nhận diện:

- Bài toán yêu cầu tìm cặp phần tử thỏa mãn một phương trình/bất phương trình (ví dụ: $A[i] + A[j] = X$).
- Mảng đầu vào đã được sắp xếp hoặc thứ tự ban đầu của mảng không quan trọng (có thể tự sắp xếp với chi phí $O(N \log N)$).

1.2. Hai con trỏ cùng chiều - Cửa sổ trượt (Sliding Window)

Cả hai con trỏ L và R cùng xuất phát từ đầu mảng. Con trỏ R di chuyển về phía trước để mở rộng "cửa sổ" (window). Khi cửa sổ vi phạm một điều kiện ràng buộc nào đó, con trỏ L sẽ được di chuyển lên để thu hẹp cửa sổ cho đến khi điều kiện được thỏa mãn trở lại.

Đặc điểm nhận diện:

- Bài toán yêu cầu tìm mảng con liên tiếp (contiguous subarray) dài nhất/ngắn nhất thỏa mãn điều kiện.
- Điều kiện của bài toán mang tính chất tích lũy và có thể thêm/bớt phần tử trong $O(1)$ hoặc $O(\log N)$.

Phân tích độ phức tạp khấu hao (Amortized Analysis): Mặc dù có một vòng lặp `while` (thu hẹp L) lồng bên trong vòng lặp `for/while` (mở rộng R), độ phức tạp không phải là $O(N^2)$. Nhận thấy rằng cả L và R đều chỉ di chuyển theo một chiều duy nhất (từ trái sang phải) và mỗi phần tử của mảng được thêm vào cửa sổ tối đa 1 lần, loại bỏ tối đa 1 lần. Do đó, số bước di chuyển tổng cộng của cả L và R không vượt quá $2N$. Độ phức tạp thời gian là $O(N)$.

2. Các bài toán điển hình

Bài toán 2.1: Two Sum (Cặp phần tử có tổng bằng X)

Phát biểu: Cho một mảng số nguyên A gồm N phần tử **đã được sắp xếp** tăng dần và số nguyên X. Hãy tìm xem có tồn tại hai chỉ số i, j $(i \neq j)$ sao cho $A[i] + A[j] = X$ hay không.

Phân tích tiếp cận: Cách ngây thơ nhất là duyệt mọi cặp (i, j) mất $O(N^2)$. Áp dụng hai con trỏ ngược chiều: Khởi tạo $L = 0$ và $R = N - 1$.

- Nếu $A[L] + A[R] = X$: Ta tìm thấy đáp án.
- Nếu $A[L] + A[R] > X$: Vì mảng đã sắp xếp, với mọi $k > L$, ta luôn có $A[k] + A[R] \geq A[L] + A[R] > X$. Do đó, phần tử $A[R]$ không thể ghép với bất kỳ phần tử nào từ đoạn $[L, R-1]$ để tạo ra tổng $\leq X$. Lựa chọn duy nhất để tổng giảm xuống là loại bỏ $A[R]$ khỏi vùng xét nghiệm, tức là $R \leftarrow R - 1$.
- Nếu $A[L] + A[R] < X$: Lập luận tương tự, phần tử $A[L]$ quá nhỏ, không thể ghép với bất kỳ ai để đạt được X, ta phải $L \leftarrow L + 1$.

Mã giả (Pseudo-code) minh họa:

```
int L = 0, R = n - 1;
while (L < R) {
    long long sum = A[L] + A[R];
    if (sum == X) return true;
    if (sum > X) R--;
    else L++;
}
return false;
```

Bài toán 2.2: Longest Subarray with Sum $\leq S$ (Cửa sổ trượt linh hoạt)

Phát biểu: Cho mảng A gồm N số nguyên **không âm** và số nguyên S. Tìm độ dài lớn nhất của một mảng con liên tiếp có tổng các phần tử không vượt quá S.

Phân tích tiếp cận: Sử dụng mô hình Sliding Window. Ta duy trì một đoạn $[L, R]$ có tổng là cur_sum. Duyệt R từ 0 đến $N-1$, mỗi bước thêm $A[R]$ vào cur_sum. Do mảng chứa số không âm, việc thêm phần tử luôn làm tổng tăng lên (hoặc giữ nguyên). Nếu $cur_sum > S$, trạng thái hiện tại bị vi phạm. Để khôi phục tính hợp lệ, ta buộc phải thu hẹp cửa sổ từ bên trái bằng cách trừ $A[L]$ khỏi cur_sum và tăng L lên, cho đến khi $cur_sum \leq S$. Lúc này, $[L, R]$ là một cửa sổ hợp lệ, ta cập nhật đáp án với $R - L + 1$.

Cài đặt mẫu:

```
int L = 0, max_len = 0;
long long cur_sum = 0;
for (int R = 0; R < n; ++R) {
    cur_sum += A[R];
    while (cur_sum > S) {
        cur_sum -= A[L];
        L++;
    }
    max_len = max(max_len, R - L + 1);
}
```

Bài toán 2.3: Số lượng phần tử phân biệt trong mỗi cửa sổ kích thước K (Cửa sổ trượt cố định)

Phát biểu: Cho mảng A độ dài N và số nguyên K. Với mỗi mảng con liên tiếp độ dài đúng bằng K, in ra số lượng giá trị phân biệt (distinct values) trong mảng con đó.

Phân tích tiếp cận: Đây là dạng bài "Fixed-size Sliding Window". Cửa sổ luôn có độ dài $R - L + 1 = K$. Khi dịch chuyển cửa sổ từ $[i, i + K - 1]$ sang $[i + 1, i + K]$, ta chỉ cần:

- **Loại bỏ** phần tử $A[i]$ (trượt khỏi cửa sổ).
- **Thêm vào** phần tử $A[i + K]$ (mới đi vào cửa sổ).

Để đếm số phần tử phân biệt $O(1)$ mỗi bước, ta dùng một cấu trúc dữ liệu `std::map` hoặc mảng tần số (Frequency Array) cùng một biến `distinct_count`. Khi thêm số x, nếu `freq[x] == 0` thì `distinct_count++`. Khi xóa số y, nếu `freq[y] == 1` thì `distinct_count--`, sau đó giảm `freq[y]`.

3. Bài tập vận dụng

Bài 1: Tổng Lớn Nhất (Fixed Window)

Tên file bài làm: `MAXWIN.cpp`

Đề bài: Cho một mảng A gồm N số nguyên và một số nguyên dương K ($K \leq N$). Hãy tìm một mảng con liên tiếp gồm đúng K phần tử sao cho tổng các phần tử trong mảng con đó là lớn nhất.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa hai số nguyên N và K ($1 \leq K \leq N \leq 10^6$).
- Dòng thứ hai chứa N số nguyên $A_1, A_2, \ldots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là tổng lớn nhất của mảng con liên tiếp gồm K phần tử.

Input	Output
7 3 1 5 -1 6 3 -2 7	10

Giải thích: Mảng con liên tiếp 3 phần tử có tổng lớn nhất là $[5, -1, 6]$ với tổng bằng 10.

Bài 2: Tổng Bằng S (Variable Window)

Tên file bài làm: `SUBSEQS.cpp`

Đề bài: Cho một mảng A gồm N số nguyên dương và một số nguyên S. Hãy kiểm tra xem có tồn tại một mảng con liên tiếp nào của A có tổng **đúng bằng** S hay không. Nếu có, hãy in ra vị trí bắt đầu và kết thúc của mảng con đó (chỉ số tính từ 1). Nếu có nhiều mảng con thỏa mãn, in ra mảng con có vị trí bắt đầu nhỏ nhất. Nếu không tồn tại, in ra -1.

Dữ liệu vào (Input):

- Dòng đầu chứa hai số nguyên N và S ($1 \leq N \leq 10^5, 1 \leq S \leq 10^{14}$).
- Dòng thứ hai chứa N số nguyên dương $A_1, A_2, \ldots, A_N$ ($1 \leq A_i \leq 10^9$).

Dữ liệu ra (Output):

- In ra hai số nguyên là chỉ số bắt đầu và kết thúc của mảng con. Nếu không tìm thấy, in ra `-1`.

Input	Output
6 12 1 3 2 5 2 1	2 5

Giải thích: Tổng đoạn từ vị trí 2 đến 5 là: $3 + 2 + 5 + 2 = 12$.

Bài 3: Trộn hai mảng đã sắp xếp (Merge Two Sorted Arrays)

Tên file bài làm: `MERGE.cpp`

Đề bài: Cho mảng A gồm N phần tử và mảng B gồm M phần tử. Cả hai mảng đều đã được sắp xếp theo thứ tự **không giảm**. Hãy áp dụng kỹ thuật hai con trỏ để trộn hai mảng này thành một mảng C gồm $N + M$ phần tử, sao cho mảng C cũng được sắp xếp theo thứ tự không giảm. Yêu cầu thuật toán hoạt động với độ phức tạp thời gian $O(N + M)$.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa hai số nguyên N và M ($1 \leq N, M \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $A_1, A_2, \ldots, A_N$ ($-10^9 \leq A_i \leq 10^9$).
- Dòng thứ ba chứa M số nguyên $B_1, B_2, \ldots, B_M$ ($-10^9 \leq B_i \leq 10^9$).

Dữ liệu ra (Output):

- In ra $N+M$ số nguyên của mảng C sau khi trộn, các số cách nhau bởi một khoảng trắng.

Input	Output
4 3 1 4 5 8 2 3 9	1 2 3 4 5 8 9

Giải thích: Sử dụng hai con trỏ i và j lần lượt duyệt qua mảng A và B. Tại mỗi bước, so sánh $A[i]$ và $B[j]$, phần tử nào nhỏ hơn sẽ được đẩy vào mảng C và tăng con trỏ tương ứng lên. Quá trình lặp lại cho đến khi cả hai mảng đều được duyệt hết.

Bài 4: Số lượng xe đạp tối thiểu (Boats to Save People)

Tên file bài làm: `BIKES.cpp`

Đề bài: Có N người cần di chuyển, người thứ i có khối lượng là W_i. Bạn cần thuê xe đạp đôi để chở tất cả mọi người. Biết rằng mỗi chiếc xe đạp chỉ có thể chở **tối đa 2 người** và tổng khối lượng của những người trên xe không được vượt quá mức tải trọng cho phép là C. Hãy tìm số lượng xe đạp ít nhất cần thiết để chở tất cả N người.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa hai số nguyên N và C ($1 \leq N \leq 10^5, 1 \leq C \leq 10^9$).

- Dòng thứ hai chứa N số nguyên $W_1, W_2, \ldots, W_N$ ($1 \leq W_i \leq C$).

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là số lượng xe đạp tối thiểu cần dùng.

Input	Output
4 3 3 2 2 1	3

Bài 5: Vòng Cổ Đa Sắc (Subarray Distinct Values)

Tên file bài làm: `UNIQUE_SUB.cpp`

Đề bài: Tom đang dạo bước trên bãi biển và nhặt được một chuỗi N viên sỏi nhiều màu sắc được xếp thành một hàng ngang. Mỗi màu sắc của viên sỏi được đại diện bởi một số nguyên dương. Tom muốn cắt ra một đoạn sỏi liên tiếp để xâu thành một chiếc vòng cổ. Là một người theo đuổi sự hoàn hảo, Tom yêu cầu chiếc vòng cổ của mình **không được có bất kỳ hai viên sỏi nào cùng màu** (mỗi giá trị chỉ được xuất hiện tối đa một lần).

Hãy đếm xem có tổng cộng bao nhiêu cách cắt ra một đoạn sỏi liên tiếp (có độ dài lớn hơn 0) thỏa mãn điều kiện khắt khe này.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên $A_1, A_2, \ldots, A_N$ ($1 \leq A_i \leq 10^9$) thể hiện màu sắc của các viên sỏi.

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là tổng số cách chọn đoạn sỏi thỏa mãn. *(Lưu ý: Đáp án có thể rất lớn, hãy sử dụng kiểu dữ liệu số nguyên 64-bit như* `long long` *trong C++).*

Input	Output
3 3 2 3	5

Giải thích: Có tổng cộng 5 đoạn sỏi liên tiếp hợp lệ bao gồm:

- 3 đoạn có độ dài 1: [3] (vị trí 1), [2] (vị trí 2), [3] (vị trí 3).
- 2 đoạn có độ dài 2: $[3, 2]$ và $[2, 3]$.

Đoạn $[3, 2, 3]$ không thỏa mãn vì viên sỏi màu 3 bị lặp lại 2 lần.

Bài 6: Hai Đội Dự Án (Two Teams)

Tên file bài làm: `TWO_TEAMS.cpp`

Đề bài: Đức Cống đang trong quá trình chiêu mộ nhân tài để thành lập studio phát triển game của riêng mình. Có N lập trình viên đến ứng tuyển, người thứ i có điểm kỹ năng là A_i. Để chuẩn bị cho một dự án lớn, Đức cần lập ra **đúng hai đội** làm việc độc lập.

Để đảm bảo tinh thần teamwork và không ai bị "khớp" khi làm việc chung, Đức đưa ra một quy tắc khắt khe: Trong cùng một đội, mức chênh lệch kỹ năng giữa người giỏi nhất và người yếu nhất không được vượt quá K. Một người chỉ có thể tham gia tối đa một đội, và một đội có thể có số lượng thành viên tùy ý (thậm chí là 1 người).

Vì muốn studio có quy mô lớn nhất có thể, hãy giúp Đức tính xem có thể tuyển **tối đa bao nhiêu nhân viên** vào hai đội này.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa hai số nguyên N và K ($1 \leq N \leq 2 \times 10^5, 0 \leq K \leq 10^9$).
- Dòng thứ hai chứa N số nguyên $A_1, A_2, \ldots, A_N$ ($1 \leq A_i \leq 10^9$) thể hiện điểm kỹ năng của các lập trình viên.

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là tổng số nhân viên tối đa có thể được chọn vào cả hai đội.

Input	Output
7 2 1 5 3 2 8 7 8	6

Giải thích: Ta có thể chọn:

- Đội 1: Chọn các nhân viên có kỹ năng $\{1, 2, 3\}$ (độ chênh lệch $3 - 1 = 2 \leq K$). Số lượng: 3.
- Đội 2: Chọn các nhân viên có kỹ năng $\{7, 8, 8\}$ (độ chênh lệch $8 - 7 = 1 \leq K$). Số lượng: 3.

Tổng số nhân viên tối đa là $3 + 3 = 6$. Nhân viên có kỹ năng 5 sẽ không được chọn.

Bài 7: Quy Hoạch Bản Đồ (Matrix Exploration)

Tên file bài làm: `MAX_BASE.cpp`

Đề bài: Tiếp tục dự án game chiến thuật của mình, Đức Cống đang thiết kế hệ thống sinh bản đồ tự động. Bản đồ được biểu diễn dưới dạng một lưới tọa độ hình chữ nhật gồm N dòng và M cột. Mỗi ô trên lưới mang một trong hai giá trị: 0 (tương ứng với đất trống) hoặc 1 (tương ứng với hang ổ quái vật).

Để tạo khu vực "Tân thủ thôn" (vùng an toàn cho người chơi mới), Đức cần khoanh vùng một ma trận con (một hình chữ nhật gồm các ô liên tiếp) có diện tích lớn nhất có thể. Tuy nhiên, để người chơi có chút thử thách nhẹ nhàng nhưng không bị choáng ngợp, khu vực này được phép chứa **tối đa** K hang ổ quái vật.

Hãy lập trình giúp Đức tìm diện tích lớn nhất của "Tân thủ thôn" thỏa mãn điều kiện trên.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa ba số nguyên N, M và K ($1 \leq N, M \leq 500$, $0 \leq K \leq N \times M$).
- N dòng tiếp theo, mỗi dòng chứa M số nguyên (chỉ gồm 0 và 1) mô tả bản đồ.

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là diện tích lớn nhất của ma trận con tìm được.

Input	Output
3 3 1 0 0 0 1 0 1 0 0 0	6

Giải thích: Ta có thể chọn ma trận con bao gồm cột 1 và cột 2, trải dài từ hàng 1 đến hàng 3 (theo chỉ số 1-based). Các giá trị trong vùng này là:

```
0 0
0 1
0 0
```

Diện tích của vùng này là $3 \times 2 = 6$, và số lượng hang quái vật (số 1) là đúng bằng $1 \leq K$. Không có vùng nào lớn hơn thỏa mãn điều kiện.

Bài 8: Chọn Số (Minimize the Difference)

Tên file bài làm: `CHONSO.cpp`

Đề bài: Cho n dãy số, mỗi dãy gồm m phần tử. Bạn cần chọn **chính xác một phần tử** từ mỗi dãy số sao cho chênh lệch giữa phần tử có giá trị lớn nhất và phần tử có giá trị nhỏ nhất trong các số được chọn là nhỏ nhất có thể.

Dữ liệu vào (Input):

- Dòng đầu tiên gồm hai số nguyên n, m ($1 \leq n, m \leq 1000$).
- n dòng tiếp theo, mỗi dòng gồm m số nguyên biểu diễn một dãy số. Các số trong dãy có giá trị tuyệt đối không vượt quá 10^9.

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là chênh lệch nhỏ nhất có thể tìm được.

Input	Output
3 3 4 9 14 6 8 10 2 7 15	2

Giải thích: Ta chọn số 9 từ dãy thứ nhất, số 8 từ dãy thứ hai, và số 7 từ dãy thứ ba. Các số được chọn là $\{9, 8, 7\}$. Chênh lệch giữa phần tử lớn nhất (9) và phần tử nhỏ nhất (7) là $9 - 7 = 2$. Không có cách chọn nào khác mang lại chênh lệch nhỏ hơn.

Bài 9: Dãy Số Liên Tiếp (Make Array Consecutive)

Tên file bài làm: `CONSECUTIVE.cpp`

Đề bài: Cho mảng A gồm n số nguyên. Bạn được phép thực hiện thao tác thay đổi giá trị của một phần tử bất kỳ trong mảng thành một số nguyên khác. Hãy tính toán xem bạn phải thay đổi ít nhất bao nhiêu số để mảng A chỉ gồm các số nguyên liên tiếp (nghĩa là sau khi sắp xếp, chênh lệch giữa hai phần tử kề nhau luôn bằng 1).

Dữ liệu vào (Input):

- Dòng đầu tiên gồm một số nguyên n ($1 \leq n \leq 10^5$).
- Dòng thứ hai gồm n số nguyên $A_1, A_2, \ldots, A_n$ ($1 \leq A_i \leq 10^9$).

Dữ liệu ra (Output):

- In ra số lượng phần tử ít nhất cần phải thay đổi.

Input	Output
5 8 5 9 4 1	2

Giải thích: Mảng ban đầu là $[8, 5, 9, 4, 1]$. Ta có thể thay đổi số 1 thành 6 và số 9 thành 7. Mảng trở thành $[8, 5, 7, 4, 6]$. Sau khi sắp xếp lại, ta được dãy liên tiếp $[4, 5, 6, 7, 8]$. Tổng số phép thay đổi là 2, và đây là phương án tối ưu nhất.

Bonus: Bằng các kiến thức đã học trước đây, bạn có thể giải bài này bằng cách khác?

Bài 10: Đếm Dãy Con

Tên file bài làm: `COUNT_SUB.cpp`

Nguồn trích từ: https://lqdoj.edu.vn/problem/26hsg9olpmuaxuan4

Đề bài: Mỗi dãy con được gọi là dãy con liên tiếp nếu dãy con đó có dạng $a_i, a_{i+1}, a_{i+2}, \ldots, a_j$ với $1 \leq i \leq j \leq n$, là dãy con của dãy a gồm n phần tử cho trước.

Cho một dãy gồm n số nguyên dương $a_1, a_2, \ldots, a_n$ và hai số nguyên dương p, q. Người ta muốn đếm số các dãy con liên tiếp của dãy số đã cho có tổng các số lớn hơn hoặc bằng p và nhỏ hơn hoặc bằng q.

Yêu cầu: Hãy lập trình đếm số các dãy con liên tiếp thỏa mãn điều kiện bài toán.

Dữ liệu vào (Input):

- Dòng đầu ghi ba số nguyên n, p, q ($1 \leq n \leq 10^5; 1 \leq p \leq q \leq 10^9$).
- Dòng thứ hai ghi n số nguyên $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^5, i = 1, 2, \ldots, n$).

Dữ liệu ra (Output):

- Một số nguyên duy nhất là số các dãy con liên tiếp thỏa mãn có tổng các số nằm trong đoạn $[p, q]$.

Input	Output
4 3 6 1 2 3 4	5

Giải thích: Có 5 dãy con liên tiếp có tổng nằm trong đoạn $[3, 6]$ bao gồm:

- Đoạn $[1, 2]$ có tổng $1 + 2 = 3$.
- Đoạn $[3]$ có tổng 3.
- Đoạn $[4]$ có tổng 4.
- Đoạn $[2, 3]$ có tổng $2 + 3 = 5$.
- Đoạn $[1, 2, 3]$ có tổng $1 + 2 + 3 = 6$.

Bài Bổ Trợ: Kỹ Thuật Tìm Phần Tử Thỏa Mãn Điều Kiện Gần Nhất

Dạng bài tìm phần tử đầu tiên bên trái/phải lớn hơn hoặc nhỏ hơn một giá trị cho trước là một bài toán nền tảng, xuất hiện cực kỳ thường xuyên trong các bài toán tối ưu trên mảng. Kỹ thuật này là tiền đề để giải quyết các bài toán phức tạp hơn như: Hình chữ nhật lớn nhất trong biểu đồ (Histogram), Mảng con có Min/Max thỏa mãn điều kiện, v.v.

Bài toán minh họa: Cho mảng A gồm N phần tử (1-based index). Với mỗi phần tử A_i $(1 \leq i \leq N)$, hãy tìm:

- L_i: Chỉ số của phần tử **đầu tiên bên trái** A_i thỏa mãn $A[L_i] \leq A[i]$. Nếu không tồn tại, $L_i = -1$.
- R_i: Chỉ số của phần tử **đầu tiên bên phải** A_i thỏa mãn $A[R_i] \geq A[i]$. Nếu không tồn tại, $R_i = -1$.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa số nguyên N $(1 \leq N \leq 10^5)$.
- Dòng thứ hai chứa N số nguyên $A_1, A_2, \ldots, A_N$ $(1 \leq A_i \leq 10^9)$.

Dữ liệu ra (Output):

- In ra N dòng, dòng thứ i chứa hai số nguyên L_i và R_i.

Input	Output
5 4 2 5 3 5	-1 3 -1 3 2 5 2 5 4 -1

Cách 1: Sử dụng Ngăn xếp đơn điệu (Monotonic Stack)

Tư duy thuật toán: Để tìm L_i, ta cần một cấu trúc dữ liệu lưu trữ các ứng viên tiềm năng ở bên trái của i. Khi xét đến A_i, nếu một phần tử $A[j]$ $(j < i)$ có giá trị lớn hơn A_i, thì $A[j]$ sẽ **vĩnh viễn không bao giờ** là đáp án cho bất kỳ phần tử nào nằm bên phải i. (Bởi vì A_i vừa nằm gần hơn, vừa có giá trị nhỏ hơn, nên A_i là một ứng viên "tốt hơn" $A[j]$). Do đó, ta duy trì một ngăn xếp (Stack) chứa chỉ số các phần tử sao cho giá trị của chúng tạo thành một dãy tăng dần (Ngăn xếp đơn điệu tăng). Cấu trúc `std::vector` hoặc `std::deque` trong C++ hoàn toàn có thể dùng thay thế `std::stack`.

```cpp
// Tim L[i]: Phan tu dau tien ben trai  <= A[i]
vector<int> st; // Luu chi so
for (int i = 1; i <= N; ++i) {
    while (!st.empty() && A[st.back()] > A[i]) {
        st.pop_back(); // Loai bo cac ung vien vo dung
    }
    L[i] = st.empty() ? -1 : st.back();
    st.push_back(i);
}

// Tim R[i]: Phan tu dau tien ben phai >= A[i]
st.clear();
for (int i = N; i >= 1; --i) {
    while (!st.empty() && A[st.back()] < A[i]) {
        st.pop_back();
    }
    R[i] = st.empty() ? -1 : st.back();
    st.push_back(i);
}
```

Độ phức tạp: Thời gian $O(N)$ vì mỗi phần tử được đẩy vào và lấy ra khỏi Stack tối đa đúng 1 lần. **Cách 2: Kỹ thuật nhảy chỉ số (Array Jumping)**

Tư duy thuật toán: Đây là một kỹ thuật cực kỳ tinh tế, tận dụng trực tiếp kết quả đã tính toán trước đó (tương tự như ý tưởng của Quy hoạch động hoặc thuật toán KMP). Giả sử ta đang đứng ở vị trí i và muốn tìm L_i. Ta bắt đầu kiểm tra phần tử liền kề bên trái là $pos = i - 1$.

- Nếu $A[pos] \leq A[i]$, ta tìm thấy đáp án ngay lập tức.
- Nếu $A[pos] > A[i]$, ta **không cần lùi từng bước** về $pos - 1, pos - 2, \ldots$ thay vào đó, ta biết chắc chắn rằng mọi phần tử nằm giữa pos và $L[pos]$ đều lớn hơn $A[pos]$ (và do đó cũng lớn hơn A_i). Ta có thể "nhảy" thẳng đến vị trí $L[pos]$ bằng lệnh `pos = L[pos]`.

```cpp
// Khoi tao mang bien
A[0] = -2e9; // Linh canh trai
A[N + 1] = 2e9; // Linh canh phai

// Tim L[i]:
for (int i = 1; i <= N; ++i) {
    int pos = i - 1;
    while (pos != 0 && A[pos] > A[i]) {
        pos = L[pos]; // Nhay tat qua cac doan khong thoa man
    }
    L[i] = (pos == 0) ? -1 : pos;
}

// Tim R[i]:
for (int i = N; i >= 1; --i) {
    int pos = i + 1;
    while (pos != N + 1 && A[pos] < A[i]) {
        pos = R[pos];
    }
```

```
    R[i] = (pos == N + 1) ? -1 : pos;
}
```

Độ phức tạp: Nhờ việc nhảy qua các phần tử đã được xác định là lớn hơn (hoặc nhỏ hơn), mỗi phần tử sẽ bị bỏ qua tối đa một lần trong suốt quá trình chạy. Kỹ thuật này cho thời gian khấu hao là $O(N)$ và **chỉ yêu cầu** $O(1)$ bộ nhớ phụ trợ (nếu không tính mảng lưu kết quả L và R).

Lưu ý thực chiến: Cách 1 (Stack) được sử dụng phổ biến và an toàn hơn trong phòng thi. Tuy nhiên, Cách 2 (Jumping) lại tỏa sáng trong các bài toán giới hạn bộ nhớ cực kỳ khắt khe, nơi việc khai báo thêm một Stack/Vector cũng có thể gây ra tràn bộ nhớ (Memory Limit Exceeded).

Bài 11: Dãy Số Đối Xứng (Palindrome Array)

Tên file bài làm: `PALIN.cpp`

Đề bài: Cho một dãy gồm N số nguyên dương. Bạn cần biến đổi dãy này thành một dãy đối xứng (đọc từ trái sang phải hay từ phải sang trái đều giống nhau).

Trong một thao tác, bạn có thể chọn hai phần tử kề nhau bất kỳ và gộp chúng lại thành một phần tử mới có giá trị bằng tổng của hai phần tử ban đầu. Hãy tính số thao tác gộp tối thiểu cần thực hiện để biến dãy ban đầu thành một dãy đối xứng.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa số nguyên dương N ($1 \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên dương $a_1, a_2, \ldots, a_N$ ($1 \leq a_i \leq 10^6$).

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là số lần gộp tối thiểu cần thực hiện.

Input	Output
5 3 10 6 4 3	1
3 2 1 4	2

Giải thích:

- **Test 1:** Ta chọn gộp hai phần tử 6 và 4 lại với nhau. Dãy trở thành $[3, 10, 10, 3]$, đây là dãy đối xứng. Cần 1 thao tác.
- **Test 2:** Đầu tiên gộp 2 và 1, dãy trở thành $[3, 4]$ (tốn 1 thao tác). Tiếp tục gộp 3 và 4, dãy trở thành $[7]$ (tốn thêm 1 thao tác). Một phần tử duy nhất luôn là dãy đối xứng. Tổng cộng cần 2 thao tác.

Bài 12: Sắp xếp sách (Shortest Unsorted Continuous Subarray)

Tên file bài làm: `ISEKAI_SORT.cpp`

Đề bài: Tại Thư Viện Hoàng Gia của Học Viện Phép Thuật, các cuốn ma đạo thư trên kệ khổng lồ vốn luôn được sắp xếp nghiêm ngặt theo mức độ ma lực (tăng dần từ trái sang phải) để tránh gây ra các phản ứng nổ dây chuyền. Tuy nhiên, hôm nay cô nàng hậu đậu Aqua đã vô tình vấp ngã, làm lộn xộn một đoạn liên tiếp các cuốn sách trên kệ A gồm N cuốn.

Để tránh bị hội học sinh kỷ luật, Kazuma quyết định dùng kỹ năng *"Time Alter"* (Đảo ngược thời gian) để tự động sắp xếp lại đoạn sách bị lộn xộn. Tuy nhiên, vì lượng MP (Mana Points) có hạn, cậu chỉ có thể niệm phép lên một **đoạn sách liên tục ngắn nhất** $(A_L, \ldots, A_R)$. Đảm bảo rằng chỉ cần đoạn sách này được sắp xếp lại cho đúng thứ tự, toàn bộ kệ sách sẽ ngay lập tức trở về trạng thái hoàn hảo (ma lực không giảm).

Hãy giúp Kazuma tìm chỉ số L và R (tính từ 1) của đoạn sách cần niệm phép. Nếu kệ sách vốn dĩ đã được sắp xếp sẵn (Aqua may mắn không làm hỏng gì), hãy in ra `-1 -1`.

Dữ liệu vào (Input):

- Dòng đầu tiên đưa vào số lượng bộ test T $(1 \leq T \leq 100)$.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng:
  - Dòng đầu tiên đưa vào N $(1 \leq N \leq 10^6)$ là số cuốn ma đạo thư.
  - Dòng tiếp theo là N số nguyên A_i $(1 \leq i \leq N, 1 \leq A_i \leq 10^7)$ thể hiện mức độ ma lực của từng cuốn. Các số cách nhau một khoảng trắng.

Lưu ý: Tổng N của tất cả các test case thường không vượt quá 10^6 để đảm bảo thời gian chạy.

Dữ liệu ra (Output):

- Đưa ra kết quả mỗi test theo từng dòng gồm hai số L và R. Nếu dãy đã được sắp xếp sẵn, in ra `-1 -1`.

Input	Output
1 11 10 12 20 30 25 40 32 31 35 50 60	4 9

Giải thích: Kệ sách có các giá trị ma lực: $10, 12, 20,$ **$30, 25, 40, 32, 31, 35$**$, 50, 60$. Đoạn bị lộn xộn nằm từ vị trí 4 (số 30) đến vị trí 9 (số 35). Nếu ta sắp xếp lại đoạn in đậm này thành $25, 30, 31, 32, 35, 40$, toàn bộ kệ sách sẽ trở thành một dãy tăng dần hợp lệ.

Bài 13: Âm Dương Độn Giáp Trận (Alternating Subarray)

Tên file bài làm: `ALTERNATING.cpp`

Đề bài: Nhập vào dãy A có N phần tử. Hãy tính độ dài của đoạn con liên tiếp dài nhất sao cho các phần tử trong đoạn con đó có dấu (âm và dương) đan xen nhau.

Lưu ý: Hai phần tử kề nhau được coi là đan xen dấu nếu một số là số dương và số còn lại là số âm. Số 0 không mang dấu dương cũng không mang dấu âm.

Dữ liệu vào (Input):

- Dòng đầu tiên chứa số nguyên dương n $(1 \leq n \leq 10^5)$.

- Dòng thứ hai chứa n số nguyên $a_1, a_2, \ldots, a_n$ ($-10^6 \leq a_i \leq 10^6$).

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là độ dài của đoạn con đan xen dài nhất tìm được.

Input	Output
8 1 2 2 -2 3 -4 3 3	5

Giải thích: Đoạn con liên tiếp dài nhất có dấu đan xen là $[2, -2, 3, -4, 3]$, nằm từ vị trí thứ 2 đến vị trí thứ 6 trong mảng ban đầu. Độ dài của đoạn này là 5.

Bài 14: Xâu An Toàn (Safe Substrings)

Tên file bài làm: `SAFESTR.cpp`

Đề bài: Cho một xâu ký tự S có độ dài N chỉ gồm các chữ cái tiếng Anh in thường.

Ta định nghĩa *độ khó chịu* $F(W)$ của một xâu W là tổng bình phương số lần xuất hiện của từng ký tự trong xâu đó. Ví dụ: Với W = "hello", ta có 2 chữ 'l', 1 chữ 'h', 1 chữ 'e', 1 chữ 'o'. Do đó $F(W) = 2^2 + 1^2 + 1^2 + 1^2 = 7$.

Một xâu được gọi là "an toàn" nếu độ khó chịu của nó không vượt quá một số nguyên K cho trước ($F(W) \leq K$). Hãy đếm số lượng xâu con liên tiếp của S là xâu an toàn. Hai xâu con được coi là khác nhau nếu chúng nằm ở hai vị trí khác nhau trong xâu S ban đầu.

Dữ liệu vào (Input):

- Dòng thứ nhất chứa hai số nguyên N và K ($1 \leq N \leq 2 \times 10^6, 1 \leq K \leq 10^{18}$).
- Dòng thứ hai chứa xâu S độ dài N.

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là số lượng xâu con an toàn của S.

Giới hạn (Subtasks):

- **Subtask 1 (30% số điểm):** $N \leq 5000$.
- **Subtask 2 (70% số điểm):** Không có ràng buộc gì thêm.

Input	Output
9 10 aabbabaaa	30
12 12 mmississippi	50

Bài 15: Kênh Truyền Tín Hiệu (Signal Routing)

Tên file bài làm: `MEDIAN_MAX.cpp`

Đề bài: Để thiết lập một mạng lưới viễn thông, người ta xây dựng N trạm phát sóng nằm trên một đường thẳng. Trạm phát sóng thứ i có cường độ phát là một số nguyên dương A_i.

Một "kênh truyền dữ liệu" hợp lệ yêu cầu phải kết nối **ít nhất** K trạm phát sóng liên tiếp nhau. Để đánh giá chất lượng của một kênh truyền gồm M trạm phát sóng ($M \geq K$), các kỹ sư sử dụng chỉ số **Cường độ lõi**. Chỉ số này được tính bằng cách: Ghi nhận cường độ của M trạm phát sóng trong kênh, sắp xếp các giá trị này theo thứ tự không giảm, và chọn ra giá trị nằm ở vị trí thứ $\lfloor (M+1)/2 \rfloor$ (vị trí tính từ 1).

Yêu cầu: Trong số tất cả các kênh truyền dữ liệu hợp lệ (gồm ít nhất K trạm liên tiếp), hãy tìm kênh truyền có **Cường độ lõi** lớn nhất và in ra giá trị đó.

Dữ liệu vào (Input):

- Dòng thứ nhất chứa hai số nguyên dương N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng thứ hai chứa N số nguyên dương $A_1, A_2, \ldots, A_N$ ($1 \leq A_i \leq N$).

Dữ liệu ra (Output):

- In ra một số nguyên duy nhất là Cường độ lõi lớn nhất tìm được.

Input	Output
5 3 4 1 5 3 2	4

Giải thích: Ta xét các kênh truyền có độ dài $M \geq 3$:

- Kênh $[4, 1, 5]$ ($M = 3$): Sắp xếp thành $[1, 4, 5]$. Vị trí lõi là $(3+1)/2 = 2$. Giá trị lõi là 4.
- Kênh $[1, 5, 3]$ ($M = 3$): Sắp xếp thành $[1, 3, 5]$. Giá trị lõi là 3.
- Kênh $[4, 1, 5, 3]$ ($M = 4$): Sắp xếp thành $[1, 3, 4, 5]$. Vị trí lõi là $\lfloor (4+1)/2 \rfloor = 2$. Giá trị lõi là 3.
- Kênh $[4, 1, 5, 3, 2]$ ($M = 5$): Sắp xếp thành $[1, 2, 3, 4, 5]$. Vị trí lõi là 3. Giá trị lõi là 3.

Cường độ lõi lớn nhất đạt được là 4.